

FactFormer

Factorized Transformer for Operator Learning

1 Introduction

FactFormer is a transformer-based neural operator architecture designed to learn mappings between functions. Its central idea is to replace the quadratic cost of conventional global attention with a factorized attention mechanism.

Let

$$\mathcal{G} : \mathcal{A} \rightarrow \mathcal{U} \tag{1}$$

be the operator that maps an input function $a(x)$ to an output function $u(x)$. After discretization on N spatial locations,

$$a(x) \longrightarrow \{a(x_i)\}_{i=1}^N, \tag{2}$$

the operator-learning problem becomes the learning of a transformation between sets of function values.

FactFormer uses attention to model long-range interactions between these locations while exploiting factorization to reduce computational cost.

2 Operator Learning Representation

Let

$$X = \{x_1, \dots, x_N\} \tag{3}$$

be the spatial discretization and let

$$a_i = a(x_i) \tag{4}$$

denote the input field.

The input feature at location x_i is

$$z_i^0 = P_{\text{in}}[a_i; x_i]. \tag{5}$$

The resulting feature matrix is

$$Z^0 \in \mathbb{R}^{N \times d}. \tag{6}$$

The transformer then learns

$$Z^0 \rightarrow Z^1 \rightarrow \dots \rightarrow Z^L. \tag{7}$$

Finally,

$$\hat{u}_i = P_{\text{out}}(z_i^L). \tag{8}$$

3 Standard Self-Attention

For an input matrix

$$Z \in \mathbb{R}^{N \times d}, \quad (9)$$

define

$$Q = ZW_Q, \quad (10)$$

$$K = ZW_K, \quad (11)$$

$$V = ZW_V. \quad (12)$$

The standard attention operation is

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (13)$$

Since

$$QK^\top \in \mathbb{R}^{N \times N}, \quad (14)$$

the memory and computational cost scales quadratically with N :

$$\mathcal{O}(N^2 d). \quad (15)$$

For high-resolution PDE discretizations, this becomes expensive.

4 Factorized Attention

FactFormer avoids explicitly constructing the full $N \times N$ attention matrix.

The key idea is to factorize the attention interaction into lower-dimensional components.

For a spatial point represented by coordinates

$$x = (x_1, \dots, x_d), \quad (16)$$

the attention kernel can be represented in factorized form:

$$K(x, y) \approx \sum_{r=1}^R f_r(x) g_r(y). \quad (17)$$

Here R is the factorization rank.

The corresponding operator is

$$(\mathcal{K}v)(x) = \int K(x, y) v(y) dy. \quad (18)$$

Substitution gives

$$(\mathcal{K}v)(x) \approx \sum_{r=1}^R f_r(x) \int g_r(y) v(y) dy. \quad (19)$$

This transforms the computation from pairwise interactions into a sequence of factorized contractions.

5 Tensor Representation

For a multidimensional domain,

$$x = (x_1, x_2, \dots, x_d), \quad (20)$$

the feature field can be represented as a tensor

$$Z \in \mathbb{R}^{N_1 \times \dots \times N_d \times C}. \quad (21)$$

Instead of treating all

$$N = N_1 N_2 \dots N_d \quad (22)$$

points as one sequence, factorized attention operates along individual dimensions. For example, for a two-dimensional field,

$$Z \in \mathbb{R}^{N_x \times N_y \times C}. \quad (23)$$

Attention can first be applied along x :

$$Z' = \text{Attn}_x(Z), \quad (24)$$

and then along y :

$$Z'' = \text{Attn}_y(Z'). \quad (25)$$

Thus the multidimensional interaction is factorized into lower-dimensional attention operations.

6 One-Dimensional Attention

For a sequence

$$Z \in \mathbb{R}^{N \times C}, \quad (26)$$

define

$$Q = ZW_Q, \quad (27)$$

$$K = ZW_K, \quad (28)$$

$$V = ZW_V. \quad (29)$$

The factorized kernel can be expressed as

$$A_{ij} \approx \sum_{r=1}^R q_{ir} k_{jr}. \quad (30)$$

Therefore,

$$Y_i = \sum_j A_{ij} V_j \quad (31)$$

becomes

$$Y_i \approx \sum_{r=1}^R q_{ir} \left(\sum_j k_{jr} V_j \right). \quad (32)$$

The expression inside parentheses is independent of i and can therefore be computed once.

7 Complexity

Standard attention requires

$$\mathcal{O}(N^2C) \quad (33)$$

operations.

Factorized attention instead requires approximately

$$\mathcal{O}(NRC), \quad (34)$$

where

$$R \ll N. \quad (35)$$

Thus

$$\boxed{\mathcal{O}(N^2C) \rightarrow \mathcal{O}(NRC)}. \quad (36)$$

This is important for operator learning because PDE solutions can require very large discretizations.

8 Transformer Block

A factorized transformer block consists of attention followed by a pointwise feed-forward network.

First,

$$Z' = Z + \text{FactAttn}(\text{LN}(Z)). \quad (37)$$

Then,

$$Z'' = Z' + \text{MLP}(\text{LN}(Z')). \quad (38)$$

The MLP is

$$\text{MLP}(z) = W_2 \sigma(W_1 z + b_1) + b_2. \quad (39)$$

Therefore,

$$\boxed{Z^{l+1} = Z^l + \text{FactAttn}(\text{LN}(Z^l)) + \text{MLP} \left(\text{LN} \left[Z^l + \text{FactAttn}(\text{LN}(Z^l)) \right] \right)}. \quad (40)$$

9 Operator Architecture

The complete architecture is

$$a(x) \xrightarrow{P_{\text{in}}} Z^0 \xrightarrow{\text{FactFormer}} Z^L \xrightarrow{P_{\text{out}}} \hat{u}(x). \quad (41)$$

Thus,

$$\boxed{\mathcal{G}_\theta = P_{\text{out}} \circ \text{FactFormer}_\theta \circ P_{\text{in}}}. \quad (42)$$

10 Training Objective

Given training pairs

$$\{a^{(i)}, u^{(i)}\}_{i=1}^{N_{\text{train}}}, \quad (43)$$

the supervised loss is

$$\mathcal{L} = \frac{1}{N_{\text{train}}} \sum_i \left\| \mathcal{G}_\theta(a^{(i)}) - u^{(i)} \right\|_2^2. \quad (44)$$

A relative error objective is

$$\mathcal{L}_{\text{rel}} = \frac{1}{N_{\text{train}}} \sum_i \frac{\|\hat{u}^{(i)} - u^{(i)}\|_2}{\|u^{(i)}\|_2 + \epsilon}. \quad (45)$$

11 PyTorch Reference Implementation

```
1
2 import torch
3 import torch.nn as nn
4 import torch.nn.functional as F
5
6
7 class FactorizedAttention(nn.Module):
8
9     def __init__(
10         self,
11         dim,
12         rank=32
13     ):
14         super().__init__()
15
16         self.dim = dim
17         self.rank = rank
18
19         self.q_proj = nn.Linear(
20             dim,
21             rank
22         )
23
24         self.k_proj = nn.Linear(
25             dim,
26             rank
27         )
28
29         self.v_proj = nn.Linear(
30             dim,
31             dim
32         )
33
34         self.out_proj = nn.Linear(
35             dim,
36             dim
37         )
38
39     def forward(self, x):
```

```

40
41     q = self.q_proj(x)
42     k = self.k_proj(x)
43     v = self.v_proj(x)
44
45     # Aggregate keys and values first
46     context = torch.einsum(
47         "bnr,bnd->brd",
48         k,
49         v
50     )
51
52     # Factorized attention output
53     out = torch.einsum(
54         "bnr,brd->bnd",
55         q,
56         context
57     )
58
59     normalization = (
60         k.sum(dim=1)
61         .unsqueeze(1)
62         .abs()
63         .clamp(min=1e-6)
64     )
65
66     out = out / normalization
67
68     return self.out_proj(out)
69
70
71 class FactFormerBlock(nn.Module):
72
73     def __init__(
74         self,
75         dim,
76         rank=32,
77         mlp_ratio=4
78     ):
79         super().__init__()
80
81         self.norm1 = nn.LayerNorm(dim)
82
83         self.attention = FactorizedAttention(
84             dim,
85             rank
86         )
87
88         self.norm2 = nn.LayerNorm(dim)
89
90         hidden_dim = (
91             dim * mlp_ratio
92         )
93
94         self.mlp = nn.Sequential(
95             nn.Linear(
96                 dim,
97                 hidden_dim

```

```

98         ),
99         nn.GELU(),
100         nn.Linear(
101             hidden_dim,
102             dim
103         )
104     )
105
106     def forward(self, x):
107
108         x = (
109             x
110             +
111             self.attention(
112                 self.norm1(x)
113             )
114         )
115
116         x = (
117             x
118             +
119             self.mlp(
120                 self.norm2(x)
121             )
122         )
123
124         return x
125
126
127 class FactFormer(nn.Module):
128
129     def __init__(
130         self,
131         in_channels,
132         out_channels,
133         dim=64,
134         depth=4,
135         rank=32,
136         coord_dim=2
137     ):
138         super().__init__()
139
140         self.input_projection = nn.Linear(
141             in_channels + coord_dim,
142             dim
143         )
144
145         self.blocks = nn.ModuleList([
146             FactFormerBlock(
147                 dim,
148                 rank
149             )
150             for _ in range(depth)
151         ])
152
153         self.output_projection = nn.Sequential(
154             nn.Linear(dim, dim),
155             nn.GELU(),

```

```

156         nn.Linear(
157             dim,
158             out_channels
159         )
160     )
161
162     def forward(
163         self,
164         coords,
165         input_field
166     ):
167
168         x = torch.cat(
169             [
170                 coords,
171                 input_field
172             ],
173             dim=-1
174         )
175
176         x = self.input_projection(x)
177
178         for block in self.blocks:
179             x = block(x)
180
181         return self.output_projection(x)
182
183
184 if __name__ == "__main__":
185
186     device = (
187         "cuda"
188         if torch.cuda.is_available()
189         else "cpu"
190     )
191
192     batch_size = 4
193     num_points = 1024
194
195     coords = torch.rand(
196         batch_size,
197         num_points,
198         2,
199         device=device
200     )
201
202     input_field = torch.randn(
203         batch_size,
204         num_points,
205         1,
206         device=device
207     )
208
209     model = FactFormer(
210         in_channels=1,
211         out_channels=1,
212         dim=64,
213         depth=4,

```



```

214         rank=32,
215         coord_dim=2
216     ).to(device)
217
218     output = model(
219         coords,
220         input_field
221     )
222
223     print("Input:", input_field.shape)
224     print("Output:", output.shape)

```

12 Summary

FactFormer’s defining idea is the factorization

$$K(x, y) \approx \sum_{r=1}^R f_r(x) g_r(y), \quad (46)$$

which transforms the direct quadratic attention computation into a lower-complexity sequence of factorized contractions.

The conceptual pipeline is

$$\boxed{\text{Function} \rightarrow \text{Embedding} \rightarrow \text{Factorized Attention} \rightarrow \text{MLP} \rightarrow \text{Function}}. \quad (47)$$

References

- [1] *FactFormer: Efficient Transformer for Long Sequence Forecasting*, arXiv preprint.